Importance Sampling for Environment Maps

Lawrence Winters

1 Introduction

Image-Based Lighting (IBL) is a collection of techniques that use real-world or artificial images, such as high-dynamic-range (HDR) environment maps, to illuminate objects in a 3D space. By sampling light from all directions in an environment, IBL allows for realistic shading and reflections that occur naturally from the surrounding environment. Compared to traditional point lights in a scene, IBL gives objects more physically accurate lighting interactions as it takes into consideration real world light information.

One challenge of IBL in Monte Carlo ray tracing is that uniform sampling of the environment often leads to inefficient computation. Many rays don't contribute effectively to the final pixel color, especially when large portions of the environment are dark. This inefficiency causes high variance (noise) in the ray-traced scenes, requiring many samples per pixel for the final image to converge to an accurate representation of the scene.

The goal of this project is to implement importance sampling for environment maps, following some of the concepts proposed in Bashford-Rogers', Debattista, and Chalmers' *Importance Driven Environment Map Sampling* [1]. By biasing rays toward brighter regions of the environment, this technique reduces variance and improves convergence, resulting in cleaner renders with fewer samples.

2 Proposed Solution

Importance Driven Environment Map Sampling (IDES) [1] proposes a method to improve the sampling of lights paths which will directly improve the rendering performance of IBL for bidirectional methods. IDES attempts to make two major contributions in the improvement of importance sampling by selecting an outgoing direction from the environment map and efficient sampling for starting points from lights. The core idea is to bias ray sampling toward regions of the environment that contribute the most and to create paths starting from these larger contributions for bidirectional path tracing.

```
INITIALIZATION:
begin
    Path tracing pre-pass
    Create path map for direction selection (Section 4)
    Create disk distribution for position selection
    (Section 5)
end

SAMPLING FROM LIGHTS:
begin
    Sample direction (Equation 4)
    Sample a starting position (Equation 7)
end
```

Figure 1 Pseudocode for the Algorithm taken from [1].

2.1 Direction Selection

The path tracing pre-pass involves tracing the scene from the camera through the environment and calculating the importance of the contribution when the environment map is collided with. When this intersection occurs, the luminance is calculated and used to determine the importance of this specific location. The importance value is then splatted onto a path map which is later used for properly selecting the direction in which paths should come. An example from the paper showcasing the properties of the path map is seen in **Figure 2**.

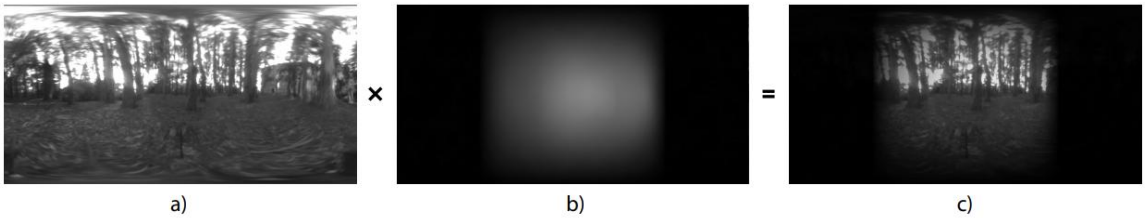

Figure 2 Image a) showcases the environment map, b) showcases the path map and c) showcases the combination of the two. Taken from [1]

2.2 Position Selection

The last method described in the paper goes over the position selection for the bidirectional path tracing. This involves choosing the starting points of the backward rays to be proportional to their contribution, which creates a density function that applies to the final color of the pixel. A six-step process is described for the position selection where weights are generated based on an incoming direction, those weights are normalized, and a piecewise constant CDF is created and sampled from to calculate the starting point of the path. The PDF is then calculated based on the sampling disk chosen and is used in the final color of the pixel. A concise visual representation is shown in **Figure 3** which was directly taken from the paper.

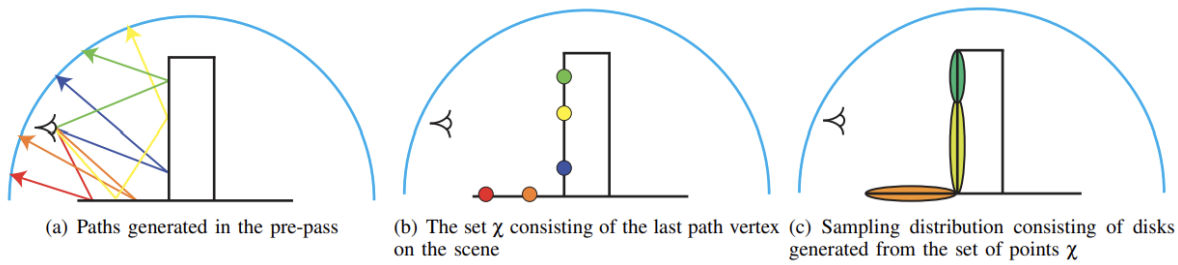

Figure 3 Portrays a quick overview of the path selection process from IDES. Taken from [1].

3 Development & Implementation

The implementation for my solution of this concept is a simplified version of the solutions talked about in the paper. Since our project is a unidirectional raytracer, I chose to keep this aspect and adjusted the final implementation as such. The first thing that changed the paper's intended implementation was that I stripped away the position selection. This aspect does not make any contribution to a unidirectional path tracer. Instead, I exclusively use the direction selection process from the paper and ensure that the direction points out to the environment map, instead of the direction going from the environment map into the scene. This is a simple adjustment made in the prepass step where I consider the viewing location of the camera rather than the entire importance map. Now that those clarifications have been made, I will start my talking about the changes to the environment sampling.

3.1 Environment Map

Since the paper assumed the environment map being sampled was spherical and my current raytracing had a cube map environment sampler, I needed to make modifications accordingly. For sampling the environment map, I changed how the texture coordinates, u and v, are calculated.

$$u = \frac{\text{atan2}(d_x, d_z)}{2\pi} + 0.5 \quad \textbf{(1)}$$

$$v = \frac{\arccos(d_y)}{\pi} \quad \textbf{(2)}$$

The coordinate u computed in **(1)** uses a glm function arc tangent which takes two arguments and will give an angle from pi to -pi using the right and forward direction of the vector d which represents the direction the ray is going. We then map it to be in the range [0, 1] through the division of 2π and the 0.5 offset. Coordinate v is calculated **(2)** using the vertical axis and arc cosine to make another coordinate in the range [0, 1]. These can now be used to find any texture coordinates on the HDR map from a world-space direction.

The next portion of code that was important for direction selection is related to the inverse sampling of the environment map. This involved converting texture coordinates to spherical coordinates and then converting those again to a direction vector using spherical to

cartesian formulas [3]. The pseudocode for this is given in **Listing 1** below.

Listing 1 Function for inverse sampling the environment map

```
glm::vec3 EnvironmentMap::InverseSample(float u, float v) const
{
  // Convert (u, v) to spherical coordinates
  float lon = (u * 2.0f * glm::pi<float>()) - glm::pi<float>();
  float lat = (1.0f - v) * glm::pi<float>();

  // Spherical to Cartesian conversion
  float sinLat = glm::sin(lat);
  glm::vec3 dir = glm::normalize(glm::vec3(sinLat * glm::sin(lon), glm::cos(lat), sinLat * glm::cos(lon)));

  // Apply inverse rotation and flip Y-axis to match texture orientation
  glm::vec3 finalDir = glm::normalize(inverseRotation * dir);
  finalDir.y *= -1.0f;
  return finalDir;
}
```

This specific function is used during the raytracing process for importance sampling where a direction is chosen based on the CDF function that I created in the direction sampler class.

3.2 Direction Sampler

The Direction Sample takes the general concepts of the paper and adjusts them to make fit in the scope of what I wanted to accomplish. The two most important concepts from this section are used for the building of my CDF and the computation of the PDF.

The CDF is straightforward and occurs directly after the luminance [2] is multiplied by the area distortion is splatted onto an important map, shown in **Listings 2** and **Listing 3**. The importance map is then unbiased using **(3)** and normalized to ensure all importance values are in the range of [0, 1).

Listing 2 Computes the Luminance of a specific color based on Rec. 709 weights

```
/*****************************************************************
* param:  color, the color to compute luminance for
*
* return: The computed luminance value
*
* brief:  Computes the luminance of a given color using standard weights
*****************************************************************/
float DirectionSampler::ComputeLuminance(const glm::vec3& color)
{
  // Set up color for luminance calculation
  glm::vec3 luminanceVec = color;

  // Apply gamma correction
  for (int i = 0; i < 3; ++i)
  {
    luminanceVec[i] = (luminanceVec[i] ≤ 0.03928) ? luminanceVec[i] / 12.92f : std::powf((luminanceVec[i] + .055f) / 1.055f, 2.4f);
  }

  // Compute luminance using Rec. 709 weights
  float luminance = luminanceVec.r * .2126f + luminanceVec.g * .7152f + luminanceVec.b * .0722f;
  return luminance;
}
```

Listing 3 Splats the luminance of a pixel onto the importance map

```
/*****************************************************************
 * param:  u, texture coordinate u
 * param:  v, texture coordinate v
 * param:  luminance, the luminance value to splat
 *
 *
 * brief:  Splat the luminance value into the importance map at the given (u, v)
 *****************************************************************/
void DirectionSampler::SplatImportance(float u, float v, float luminance)
{
  // Convert UV to pixel coords
  int px = glm::clamp(int(u * width), 0, width - 1);
  int py = glm::clamp(int(v * height), 0, height - 1);

  // Index of the pixel
  int index = py * width + px;

  // Texels cover different amounts of area, include sin of theta
  float theta = v * glm::pi<float>();

  // Accumulate importance (luminance)
  importanceMap[index] += luminance * glm::sin(theta);
}
```

$$\varepsilon_l = \frac{\delta}{|NZ|}\sum_{\theta\in\vartheta}\hat{C}(\omega_\theta) \quad \textbf{(3)}$$

This unbiasing equation involves a small value epsilon of .001, the nonzero value and the accumulated approximation of the importance of the camera over all directions from the splatting process. This added to every zero pixel to ensure the importance map is unbiased.

After the importance map is finished being normalized, The CDF is built by accumulating the importance map over the entire range of pixels. A binary search is conducted, where we determine what random texel we are sampling after selecting a random value between [0, 1]. Resource [4] contains **(4)**, which is used for the sampling of the infinite area light and allows for the calculation of the probability of choosing this texel based on the solid angle measured on the unit sphere.

$$pdf = importance(u,v)\cdot\frac{W*H}{2\pi^2 sin\theta} \quad \textbf{(4)}$$

After implementing the proper importance of sampling based on directions, I had to make one final adjustment to the existing code. These adjustments take place in my shading and material functions, where I add a PDF and BRDF element for the diffuse surfaces so that direct lighting and indirect lighting properly affect the objects in the scene.

3.3 Pixel Shading

The PDF is found by taking the cosine of theta, or the angle between the normal and the new direction, divided by pi and similarly for the Lambertian BRDF by taking the albedo color normalized over pi. All of this is seen in our improved diffuse shading function in **Listing 4** and the new addition of the direct lighting within **Listing 5**.

Listing 4 Modified shading function including PDF and BRDF for return

```
Material::ShadingInfo Diffuse::Shade(const Ray& ray, const ObjectManager::IntersectData& data, const glm::vec3& intersectionPoint) const
{
  glm::vec3 n = data.normal;

  if (glm::dot(ray.direction, data.normal) > 0)
  {
    n = -data.normal;
  }

  glm::vec3 origin = intersectionPoint + (n * EPSILON);

  glm::vec3 randDir = MonteCarloMethod();
  glm::vec3 newDir = randDir + n;

  // cos(theta) / pi -> cosine-weighted hemisphere PDF
  float pdf = glm::dot(newDir, n) / glm::pi<float>();

  // Construct a new ray with that direction, offset by a small epsilon to avoid self-intersection
  Ray newRay(origin, newDir);

  // Addition of the pdf and wl in the shading info for MIS
  return ShadingInfo(color, newRay, pdf, color / glm::pi<float>());
}
```

Listing 5 Improved bouncing of rays to include direct lighting

```
  // Perform Direct lighting
  glm::vec3 Ld(0.0f);

  if (Configuration::ENABLE_IS)
  {
    // Importance sample the environment
    DirectionSampler::ImportanceSample sample = dirSampler->SampleImportance(envMap);

    if (sample.pdf > 0.0f)
    {
      float cosTheta = glm::dot(intersect.normal, sample.dir);
      if (cosTheta > 0.0f)
      {
        // Shadow ray
        glm::vec3 shadowOrigin = hitPos + intersect.normal * EPSILON;
        Ray shadowRay(shadowOrigin, sample.dir);
        auto shadowHit = objectManager.CheckRayIntersection(shadowRay);

        // If not occluded
        if (!shadowHit.object)
        {
          // Environment radiance in that direction
          glm::vec3 Le = envMap->Sample(sample.dir);

          // MIS weight (power heuristic)
          float envPDF2 = sample.pdf * sample.pdf;
          float w = envPDF2 / (envPDF2 + shading.pdf * shading.pdf);

          // Direct L contribution
          Ld = shading.wl * Le * cosTheta * (w / sample.pdf);
        }
      }
    }
  }

  // Sum contributions
  return Li + Ld;
```

For direct lighting we find a direction from the environment's importance sample with the attached PDF. If the direction ω_s is un-occluded, we accumulate the per-sample estimator as seen in **(5)**.

$$\hat{L}_d = \frac{f(\omega_i,\omega_s)\, L_e(\omega_s) \cos\theta\, w(\omega_s)}{p_{\text{env}}(\omega_s)} \quad \textbf{(5)}$$

where $f(\omega_i, \omega_s)$ represents the BRDF, $L_e(\omega_s)$ is the luminance over the specific area and $\cos\theta$ is the angle between the new ray and intersection normal. $w(\omega_s) = \frac{p_{\text{env}}(\omega_s)^2}{p_{\text{env}}(\omega_s)^2 + p_{\text{brdf}}(\omega_s)^2}$ is the power-heuristic MIS weight combining environment and BRDF sampling. This is all divided by the PDF of the environment map based on this new sample direction [4].

4 Demo & Explanation

The fundamentals of the ray tracer that was created for the class are almost identical to my project. The addition of the importance sampler allows my ray tracer to see improvements in the globally illuminated scenes, showing brighter scenes quicker with direct light contributions. The demo showcase will provide images of the output I generated from the final project.

4.1 Demo Showcase

The initial environment scene is sampled during the prepass where I choose to trace the importance map out to a png image which can be viewed separately. Below in **Figure 4**, you can see the transformation from the initial hdr color scene, to a heatmap of the scene in grayscale and then the importance pass is made to find the area with the largest contribution in the scene.

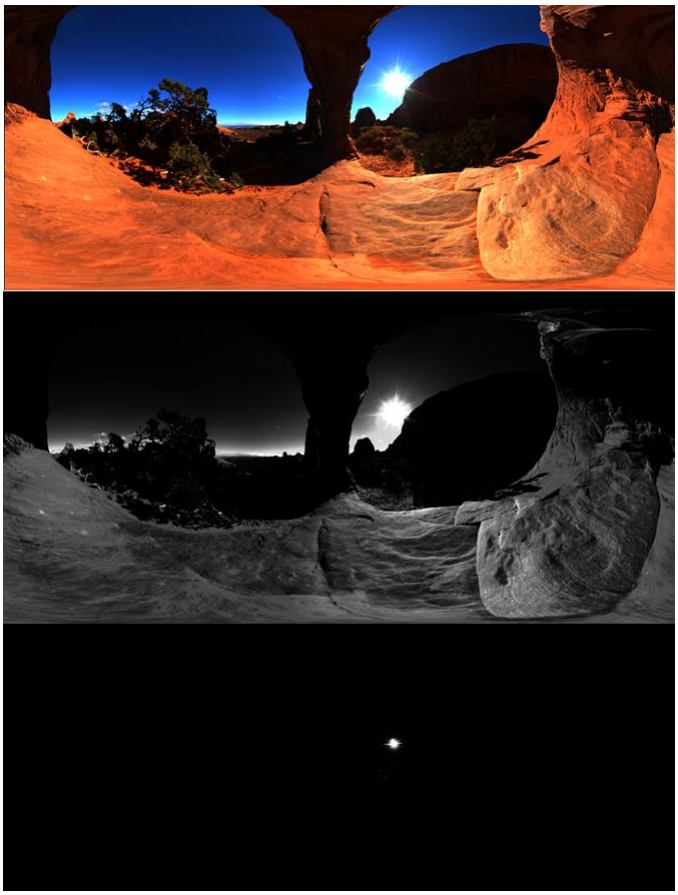

Figure 4 Showcases the importance sampling of the environment from top to bottom.

The scene in **Figure 4** has a large amount of contribution coming from the sun and when related to the other elements of the scene, it becomes the only applicable light source in the scene. Since this map has such an intense light source in one direction, it makes viewing the importance of sampling very noticeable when the sun is directly hitting our objects. When you compare the same effect in another scene, the lighting is spread over the map more uniformly, as seen in **Figure 5**.

Figure 5 Showcases the importance sampling on the Night Sea sky scene

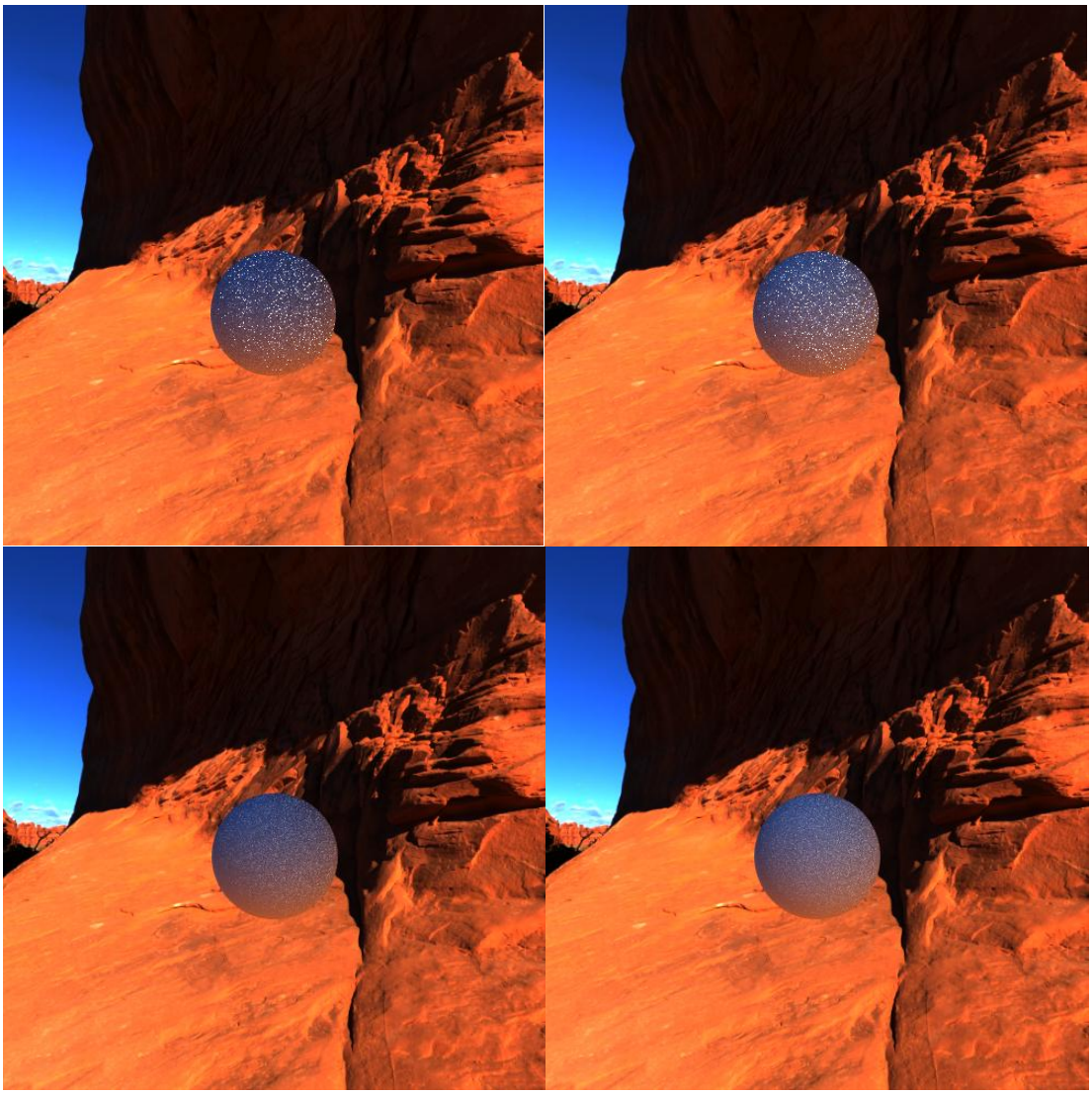

Figure 6 Top images show the uniform IS on the left versus the non-importance sampled sphere on the right at 100 samples. Bottom uses 1000 samples.

This important sampling happens over the entire scene and is useful in instances where the scene is not closed off as shown in **Figure 6**. The uniform importance sampling can make differences when the main contribution of light is near a blocked off set of objects.

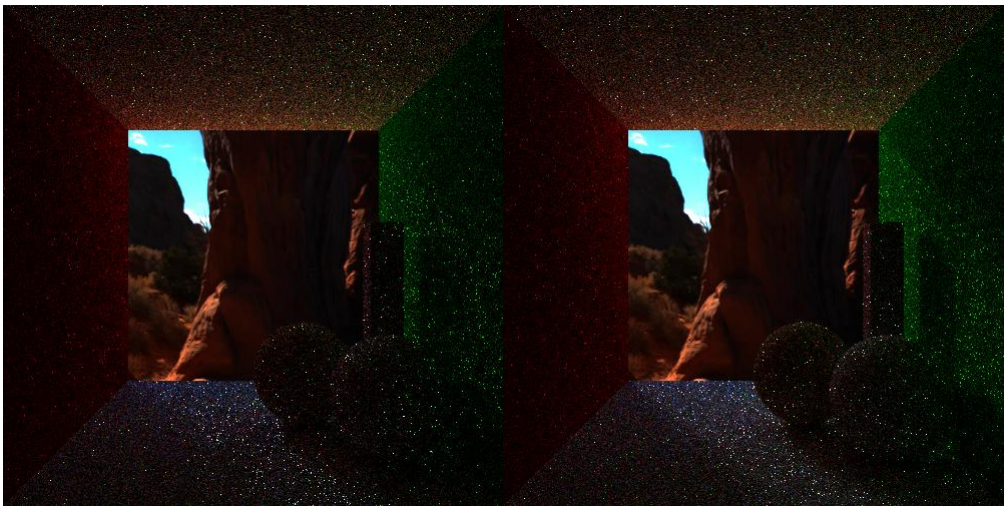

Figure 7 Non IS lighting shown on the left and IS light on the right

It's clear that importance sampling offers better illumination of the scenes when a strong lightsource is nearby as shown in **Figure 7**. What would happen if the light source is not near the closed off scene though? With this importance sampling, rays would be shot towards the sun and would never contribute anything to the environment because it's impossible to get there. However, using IDES with the prepass function, we are able to obtain an importance map and scene that illuminates better when the main source of light isn't near the entry point of the scene.

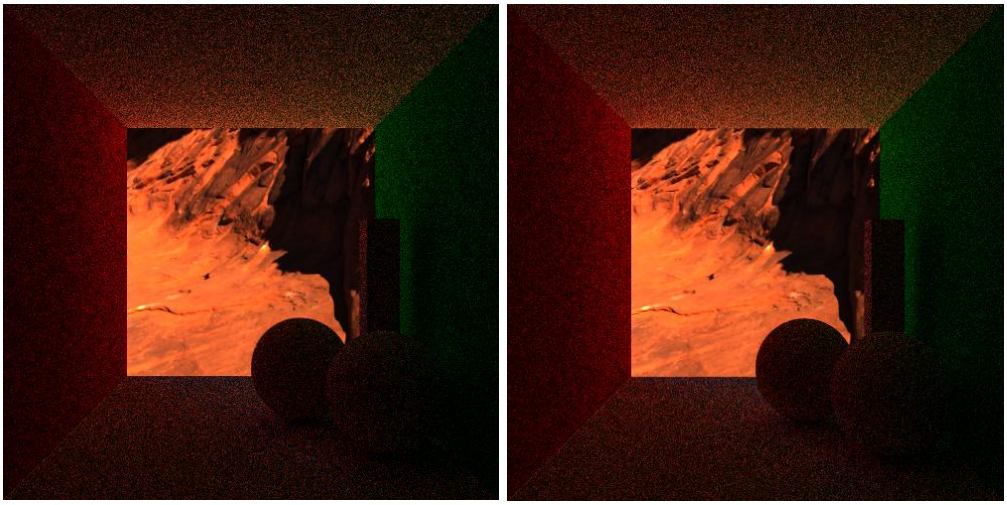

Figure 8 Uniform Importance Sampling (left) versus IDES Sampling (right).

The scene in **Figure 8** showcases the difference in importance sampling across the entire environment versus sampling according to the paper's view importance struct. It's not extremely visible as the scene is naturally dark, but the scene on the right is better lit, with all the walls showing a greater brightness and the sphere showing some color as well as a proper shadow. **Figure 9** showcase the output IDES sample for this specific scene and why it performs so much better when lighting this scene un comparison to the uniform

distribution over the entire environment map.

Figure 9 View specific importance sampling like the IDES paper.

4.2 Demo Explanation

Using the demo is very straightforward, as the fundamentals of the ray tracer stayed the same. The only difference is that there are more options for sampling the scene based on the uniform sampling of the map or the view specific sampling. All these settings can be triggered with number key presses, with number key 3 lights with no importance, 4 samples with the uniform sampling, and 5 samples with IDES. Lastly, the config file contains new settings to be able to change the number of pre-pass samples and enabling the radial splatting. Radial splatting is disabled by default because it significantly slows down the prepass step.

5 Possible Improvements

The major improvements that can be made to my current demo include adding bidirectional path tracing, extending the shading to work with surfaces other than the diffuse materials, and include importance sampling for other light sources in the scene. Bidirectional path tracing is the most valuable improvement, as it would allow my demo to gain all the benefits of the proposed solution, resulting in scenes converging faster. Extending to other materials would be simple, only needing to calculate the proper PDF and BRDF for the surfaces. Lastly, including other lights into the scene could help with better illumination but may also take away from the whole purpose of having global illumination being sampled better to produce brighter scenes.

6 Problems During Development

During the implementation of the environment map-based importance sampling system, several practical and conceptual challenges arose. These problems helped clarify why certain techniques, such as cube mapping or bidirectional path tracing, were avoided or modified.

6.1 Cube Map

One problem I encountered was the complexity of working with cube maps. Even though cube maps are conceptually just six faces, converting between directions, faces, and UV coordinates introduced a lot of edge-case issues. Small mistakes in the face-selection logic or in the mapping between direction vectors and cube faces produced incorrect sampling. These issues were difficult to debug because errors only appeared for specific directions or angles, which made the entire mapping step very sensitive.

6.2 Sensitivity to the PDF

Another major issue was how sensitive the renderer was to errors in the probability density function. Even a small mistake, such as normalizing incorrectly or forgetting a factor of $\sin\theta$ immediately caused the entire scene to appear black or extremely noisy. Since the PDF directly controls how rays are sampled, a single incorrect value can mess up the entire lighting calculation. This made debugging the sampler challenging because most symptoms looked the same, even though the underlying mistake could be anywhere in the probability computation.

6.3 Complexity of Bidirectional Path Tracing

Bidirectional path tracing added another layer of complexity. Compared to forward path tracing, BDPT requires constructing two separate paths, one from the camera and one from the scene light and then connecting them in several different ways. Each connection path has its own weight, visibility test, and MIS (multiple importance sampling) term. This was the main reason why I gave up adding this to my ray tracer rather quickly.

7 Conclusion

In this project, I implemented a unidirectional path tracer with importance sampling using spherical environment maps. Throughout development, I encountered several challenges, from the complexity of cube-map direction conversions to sensitivity in the sampling PDF. These issues showed how small mathematical mistakes can dramatically affect the rendering of the scene, resulting in black images in most cases. Despite these difficulties, the final system successfully generates brighter scenes by combining direct and indirect lighting paths based on the luminance from the environmental map. This process taught me the importance of probability handling and sampling strategies of global illumination.

8 References

[1] Bashford-Rogers, T., Debattista, K., & Chalmers, A. (2013, November). Importance Driven Environment Map Sampling. https://thomasbashfordrogers.com/Papers/IDES.pdf

[2] Colour Luminance and Contrast Ratio. 101 Computing - Boost Your Programming Skills! (2020, April 28). https://www.101computing.net/colour-luminance-and-contrast-ratio

[3] DQ, N. (n.d.). Spherical Coordinates. Spherical coordinates - Math Insight. https://mathinsight.org/spherical_coordinates

[4] Pharr, M., Jakob, W., & Humphreys, G. (2021). Physically Based Rendering. Physically Based Rendering: From Theory to Implementation. https://www.pbr-book.org/3ed-2018/contents